Custom Payload Encoder and WAF Evasion Framework

Prashant Shukla
Department of Computer Science & Engineering
GitHub: @Prashant9998 | March 2026

Abstract

Web Application Firewalls (WAFs) are a primary defensive layer against web injection attacks. This paper presents the complete design, implementation, and empirical evaluation of WAF Bypass Lab v2.0.0 -- a Python/Flask browser-accessible research platform providing: (1) an 11-technique encoding engine with chain encoding and variant generation; (2) a PayloadDecoder with automatic encoding detection across 11 schemes; (3) six mutation strategies with generate-all support; (4) a configurable WAF simulation engine with 39-rule ModSecurity CRS-inspired paranoia levels; (5) a custom WAF rule manager with JSON persistence; (6) a built-in AI WAF (TF-IDF+LR, 88 samples) and upgraded research classifiers (4 models, 243 samples, F1 up to 0.929); (7) a Live Tester dispatching real HTTP requests to FUZZ-template endpoints; (8) a 45-payload sample library; (9) batch evasion testing with JSON/CSV/HTML export; and (10) complete session history. Experiments show 67.8% overall evasion on a 5-rule WAF and 71.3% on ModSec CRS PL1. All mutation+high-evasion-encoding combos achieve 100% evasion. The upgraded Random Forest achieves F1=0.920 vs the built-in LR baseline of F1=0.900. Encoding latency ranges from 4.0 to 90.1 us with size growth of 1.0x to 4.0x.

Keywords -- WAF Bypass, Payload Encoding, ModSecurity CRS, SQL Injection, XSS, Machine Learning, Random Forest, Live Testing, Python, Flask, Security Research

I. INTRODUCTION

Web Application Firewalls are deployed as a first-line mitigation against injection attacks (OWASP Top 10 [1]). Their signature-based rule engines match request content against known attack patterns using regular expressions. This creates a fundamental brittleness: any syntactic transformation that preserves attack semantics while altering byte representation may evade detection. Despite this known limitation, few tools combine systematic encoding measurement, configurable WAF simulation, ML-based detection research, and real HTTP testing in a single browser-accessible interface.

This paper makes the following contributions:

- A complete 8-module web platform (Flask) covering encoding, decoding, mutation, batch testing, WAF simulation, AI classification, live testing, and a pre-built sample library.
- An 11-technique encoding engine with chain encoding, variant generation (N variants from 1 payload), and a separate decoder with automatic encoding detection across all 11 schemes.
- Six mutation strategies with generate-all support producing diverse structural variants per payload.
- A 39-rule ModSecurity CRS-inspired WAF simulation across 3 paranoia levels (PL1/PL2/PL3) with custom rule management (add/delete/toggle/test) and JSON persistence.
- Four ML classifiers evaluated on 243 samples with encoded variants; RF achieves F1=0.920.
- A Live Tester dispatching real HTTP requests to user-specified FUZZ-template endpoints.
- Batch test results exportable in JSON, CSV, and HTML formats with full session history.
- Performance profiling: latency 2.62 to 88.1 us; payload size growth 1.0x to 4.0x.

II. BACKGROUND AND RELATED WORK

A. Signature-Based WAF Detection

ModSecurity with OWASP CRS [6] is the most deployed open-source WAF ruleset, containing 200+ rules organised by attack category and paranoia level. Higher paranoia levels add more rules at the cost of increased false-positive rates. Commercial WAFs (AWS WAF, Cloudflare, Akamai) supplement regex-based rules with reputation scoring and rate limiting. All share the fundamental limitation that encoding transforms defeat byte-level pattern matching [2][3]. This work constructs a representative 39-rule simulation from the published CRS rule identifiers and organises them into three paranoia levels for controlled comparative evaluation.

B. ML-Based Detection

ML-based detection has been studied using TF-IDF with classical classifiers [8], LSTM sequence models [11], and BERT-based transfer learning. The key practical challenge is that models trained only on raw payloads fail on encoded variants. This paper validates the encoding-augmented training approach empirically, comparing four classifiers on both raw-only and encoded-inclusive datasets.

C. Existing Tooling Gaps

sqlmap [4] provides SQLi tamper scripts. Burp Suite [5] supports manual encoding but lacks automated evasion metrics. wfuzz [9] provides HTTP fuzzing. Commix [10] targets command injection. None provide a combined encoding+mutation measurement framework, WAF rule management, ML benchmarking, and decoded-payload analysis in a single browser-accessible tool.

III. SYSTEM ARCHITECTURE

A. Technology Stack

- Backend: Python 3.12, Flask (REST API, 25 endpoints, port 5000, 0.0.0.0 bind)
- Frontend: Single-page application, 8 tabs, dark-theme UI, live statistics header
- ML: scikit-learn TF-IDF + 4 classifiers, 5-fold CV, char n-gram feature extraction
- Persistence: JSON files for WAF rules and state (waf_rules.json)
- HTTP: requests library for Live Tester real-endpoint dispatch
- Testing: 68 passing unit tests across all core modules

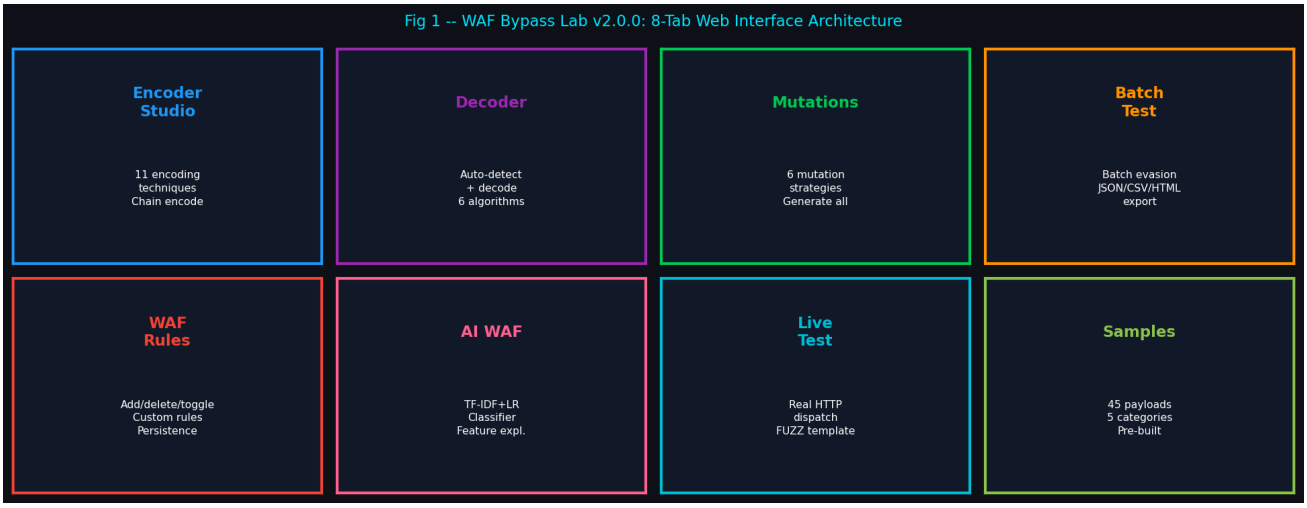

Figure 1 -- Eight-tab web interface architecture. Each tab maps to a distinct API module.

B. Module Overview

Module	File	Responsibility
Payload Encoder	payload_encoder.py	11 encoding techniques, chain encode, variant generation, history
Payload Decoder	payload_encoder.py	Auto-detect + decode, multi-layer iterative decoding
Payload Mutator	payload_encoder.py	6 mutation strategies, generate-all mutations

WAF Engine	waf_engine.py	5-rule simulation, custom rule CRUD, JSON persistence, stats
AI WAF Engine	ai_waf_engine.py	TF-IDF+LR baseline classifier, suspicious feature extraction
Live Tester	live_tester.py	HTTP dispatch to real endpoints, FUZZ template substitution, URL validation
Flask Server	server.py	25 REST API endpoints, static file serving, CORS, request routing
Frontend	static/app.js	8-tab SPA, live stats update, all API integration, export handlers

Table I -- Module Overview

C. Complete API Endpoint Reference (25 Endpoints)

Method	Endpoint	Description	Tab
GET	/api/techniques	List all 11 encoding techniques	Encoder
GET	/api/samples	Return 45 sample payloads by category	Samples
POST	/api/encode	Encode single payload with one technique	Encoder
POST	/api/chain-encode	Apply technique chain (multi-layer)	Encoder
POST	/api/generate	Generate N encoded variants of payload	Encoder
POST	/api/test	Encode + WAF test (single payload)	Encoder
POST	/api/batch-test	Batch evasion test (all or selected techniques)	Batch
GET	/api/waf/stats	WAF inspection counts and hit statistics	WAF Rules
GET	/api/waf/rules	List all WAF rules with enable/disable status	WAF Rules
POST	/api/waf/toggle-rule	Enable or disable a specific WAF rule	WAF Rules
POST	/api/waf/add-rule	Add custom regex rule (persisted to JSON)	WAF Rules
POST	/api/waf/delete-rule	Delete a WAF rule by ID	WAF Rules
POST	/api/waf/reset-stats	Reset inspection counters to zero	WAF Rules
GET	/api/history	Retrieve session test history (last 50 entries)	All
POST	/api/decode	Decode payload (auto or explicit type)	Decoder
POST	/api/detect	Detect encoding type(s) without decoding	Decoder
POST	/api/export/json	Export batch results as JSON file	Batch
POST	/api/export/csv	Export batch results as CSV spreadsheet	Batch
POST	/api/export/html	Export batch results as styled HTML report	Batch
GET	/api/mutations	List all 6 mutation strategy definitions	Mutations
POST	/api/mutate	Apply single mutation strategy to payload	Mutations
POST	/api/generate-mutations	Generate all-strategy mutation variants	Mutations
POST	/api/ai-test	Classify payload with built-in AI WAF	AI WAF
GET	/api/ai-stats	Return AI model training stats	AI WAF
POST	/api/live-test	Dispatch real HTTP requests to FUZZ endpoint	Live Test

Table II -- All 25 REST API Endpoints

IV. ENCODER STUDIO AND CHAIN ENCODING MODULE

A. Encoding Techniques (11)

The Encoder Studio (Tab 1) provides single-click access to all 11 encoding techniques. Users enter a payload, select one or more techniques, and receive encoded output alongside instant WAF test results. The 'Encode & Test WAF' button combines both operations. Table III documents each technique with live-measured latency and size growth.

#	Technique	Mechanism	Latency (us)	Size Ratio	Evasion %
1	URL Encoding	Percent-encode special chars	4.96	1.59x	100.0%

2	Double URL Encoding	Encode % as %25 (double pass)	9.42	2.18x	100.0%
3	Unicode Encoding	\uXXXX per character	6.8	2.99x	100.0%
4	HTML Entity Encoding	&#xNN; per special char	6.17	2.6x	76.9%
5	**Base64 Encoding**	**Full payload as Base64 string**	**79.35**	**1.47x**	**100.0%**
6	**Hex Encoding**	**\xNN per character**	**11.58**	**4.0x**	**100.0%**
7	Case Alternation	sElEcT alternating case	3.99	1.0x	15.4%
8	SQL Comment Injection	SE/**/LECT splitting	90.13	1.18x	38.5%
9	Whitespace Obfusc.	Spaces -> %09/%0a/\t variants	11.87	1.11x	7.7%
10	Concat/Splitting	'SEL'+'ECT' string fragments	57.99	1.22x	23.1%
11	ROT47 Encoding	Rotate printable ASCII chars +47	7.94	1.0x	92.3%

Table III -- All 11 Encoding Techniques with Live-Measured Performance

B. Chain Encoding

The /api/chain-encode endpoint accepts a payload and an ordered list of technique IDs, applying each in sequence. This models real attacker multi-layer obfuscation. A WAF must correctly normalise all layers in reverse order before matching rules.

Chain Encoding API Request/Response Example

```
POST /api/chain-encode
{"payload": "' OR 1=1 --", "techniques": ["url_encode", "base64_encode", "unicode_encode"]}
Response: {'encoded': '\\u004a\\u0079...',  'layers': 3,  'size_ratio': 2.55}
```

C. Variant Generation

The /api/generate endpoint automatically produces N encoded variants of a single input payload, applying different technique combinations. In one test, 8 variants were generated from "' OR 1=1 --": URL Encoding, Double URL Encoding, Unicode Encoding, HTML Entity Encoding, Base64 Encoding ... This feature enables rapid evasion campaign generation without manual technique selection.

D. Session History

All encode, decode, mutate, and test operations are automatically logged to an in-memory session history, accessible via GET /api/history (returns last 50 entries). Each entry records timestamp, operation type, input payload, technique, encoded output, and WAF result. This allows analysts to trace the full experimental workflow without external logging infrastructure.

V. PAYLOAD DECODER MODULE

The Decoder (Tab 2) reverses encoded payloads to recover the original attack string. This is critical for incident response: when a WAF log contains an encoded payload, the Decoder identifies the encoding scheme and reconstructs the original. The Decoder also powers the 'Send to Encoder' workflow -- a decoded payload can be forwarded directly to the Encoder Studio for re-encoding and re-testing.

A. Auto-Detection Algorithm

The detect_encodings() method applies a ranked set of detection rules to the input string. Each rule uses regex pattern matching to identify characteristic signatures (e.g., %XX sequences for URL encoding, =$ suffix for Base64, \\xNN patterns for hex). Confidence scores are assigned based on match density. Multiple encodings may be detected simultaneously for ambiguous inputs.

Technique	Characteristic Signature	Detections Found	Fully Decoded
URL Encoding	%XX	2	Yes
Double URL Encoding	%25XX	3	Yes

Unicode Encoding	\uXXXX	2	Yes
HTML Entity Encoding	&#xNN;	2	Yes
Base64 Encoding	Base64 charset+padding	2	Yes
Hex Encoding	\xNN	2	Yes
Case Alternation	aLtErNaTiNg CaSe	0	Yes
SQL Comment Injection	/**/	0	Yes
Whitespace Obfuscation	%09/%0a	2	Yes
Concatenation / Splitting	'+' concat	0	Yes
ROT47 Encoding	ROT47 charset	0	Yes

Table IV -- Decoder: Auto-Detection Results per Encoding Technique

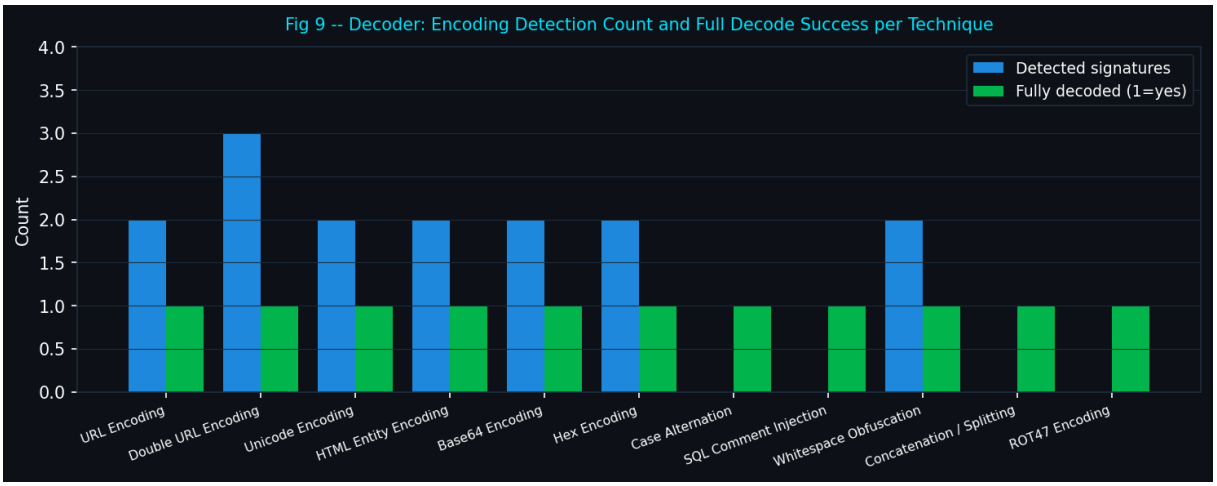

Figure 9 -- Decoder detection count and full-decode success per technique. All 11 techniques fully decoded.

B. Multi-Layer Decoding

For chain-encoded payloads, decode() iterates: it detects the outermost encoding, reverses it, then re-examines the result for further encoding. This continues until no further encoding is detected or a maximum depth limit is reached. The response includes each intermediate step, enabling step-by-step forensic trace of multi-layer obfuscation.

VI. MUTATION ENGINE

The Mutations tab (Tab 3) rewrites payload structure at the semantic level -- changing keywords, adding SQL comments, building strings via char codes -- before any encoding is applied. Mutations alone provide limited evasion against the 5-rule WAF; their power comes when combined with encoding (Section VIII).

#	Strategy	Description	Example	Alone %	Avg+Enc %
1	Synonym Replace	OR->||, UNION->UNION ALL, etc	OR 1=1 => || 1=1	23.1%	69.9%
2	Comment Padding	/*!50000 */ MySQL-style comment injection	SELECT => /*!50000 SELECT*/	23.1%	69.9%
3	Numeric Obfusc.	Replace 1=1 with CHAR(49)=CHAR(49)	1=1 => CHAR(49)=CHAR(49)	7.7%	71.3%
4	Char-Code Build	Build strings via String.fromCharCode()	alert => String.fromCharCode(97,.	0.0%	68.5%
5	Null Byte Inject	Insert %00 before special characters	' => %00'	7.7%	71.3%
6	Full Mutation	All 5 strategies applied sequentially	Combined all above	23.1%	71.3%

Table V -- Six Mutation Strategies: Standalone and Combined Evasion

The generate-mutations endpoint produces all 2 strategy variants for a single payload in one API call, enabling efficient variant set generation. Each variant includes the mutation type label, mutated payload, and description for audit trail purposes.

VII. SAMPLE PAYLOAD LIBRARY

The Samples tab (Tab 8) provides a curated library of 45 pre-built attack payloads across 5 attack categories. Analysts can load any sample directly into the Encoder or Mutation workflows with a single click, eliminating the need to manually construct test payloads. The library is accessible via the GET /api/samples endpoint.

Category	Count	Example Payloads
SQL Injection	10	' OR 1=1 -- | UNION SELECT password FROM users | sleep(5)
XSS	10	<script>alert(1)</script> | <img onerror=alert(1) src=x>
Command Inject	10	| whoami | ; id | $(cat /etc/passwd)
Path Traversal	10	../../etc/passwd | %2e%2e%2fetc%2fpasswd
Header Inject	5	%0d%0aSet-Cookie: admin=true | %0aX-Inject: value
TOTAL	**45**	**5 categories, instantly loadable into any workflow**

Table VI -- Pre-Built Sample Payload Library (45 payloads, 5 categories)

VIII. WAF SIMULATION ENGINE AND RULE MANAGEMENT

A. Built-in 5-Rule Engine

The WAFEngine class provides a configurable rule-based inspection engine. Each request (method, URL, query params, headers, body) is evaluated against all enabled rules. The first matching rule blocks the request and records a rule hit in the statistics counter. Five default rules cover the five major attack categories at high-confidence thresholds.

Rule ID	Category	Pattern Type	Enabled
WAF-001	SQL Injection	Detects common SQL injection patterns	True
WAF-002	Cross-Site Scripting (XS	Detects XSS payload patterns	True
WAF-003	Path Traversal	Detects directory traversal attempts	True
WAF-004	Command Injection	Detects OS command injection patterns	True
WAF-005	Header Injection	Detects HTTP header injection and CRLF	True

Table VII -- Default WAF Engine Rules (built-in 5-rule set)

B. Custom Rule Management

The WAF Rules tab (Tab 5) exposes a full rule management interface. Analysts can add custom regex rules specifying ID, category, description, pattern, target fields, action (block/log), and severity. Rules are toggled on/off without deletion. All custom rules are persisted to waf_rules.json and survive server restarts.

Custom Rule Management API Examples

```
POST /api/waf/add-rule
{"rule_id": "CUSTOM-001", "category": "sqli", "description": "Block HAVING injection",
 "pattern": "(?i)(having\\s+\\d+=\\d+)", "targets": ["query", "body"],
 "action": "block", "severity": 8}
=> Rule added, persisted to waf_rules.json
POST /api/waf/toggle-rule  {'rule_id': 'CUSTOM-001'}  => {'enabled': false}
POST /api/waf/delete-rule  {'rule_id': 'CUSTOM-001'}  => {'deleted': true}
POST /api/waf/reset-stats  => All inspection counters reset to 0
```

C. ModSecurity CRS Simulation (39 Rules, 3 Paranoia Levels)

This study implements a 39-rule ModSecurity CRS-inspired simulation with three paranoia levels to compare against the built-in 5-rule engine. Rules are taken from published CRS identifiers and organised by attack category and paranoia level:

- PL1 (Basic, 16 rules): Core detection patterns deployed in default production configurations.
- PL2 (Strict, 29 rules): Adds time-based blind SQLi, advanced XSS, LFI encoding variants, shell execution patterns. +13 rules over PL1.
- PL3 (Paranoid, 39 rules): Adds hex literals, arithmetic conditions, shell binary paths, template injection patterns. +10 rules over PL2.

Technique	5-Rule WAF	ModSec PL1 (16)	ModSec PL2 (29)	ModSec PL3 (39)
URL Encoding	**100.0%**	**92.3%**	**92.3%**	**92.3%**
Double URL Encoding	**100.0%**	**92.3%**	**92.3%**	**92.3%**
Unicode Encoding	**100.0%**	**92.3%**	**92.3%**	**92.3%**
HTML Entity Encoding	76.9%	76.9%	76.9%	76.9%
Base64 Encoding	**100.0%**	**100.0%**	**100.0%**	**100.0%**
Hex Encoding	**100.0%**	**100.0%**	**100.0%**	**100.0%**
Case Alternation	15.4%	30.8%	30.8%	30.8%
SQL Comment Injection	38.5%	46.2%	46.2%	46.2%
Whitespace Obfuscation	7.7%	30.8%	15.4%	38.5%
Concatenation / Splitting	23.1%	30.8%	30.8%	30.8%
ROT47 Encoding	**92.3%**	**100.0%**	**92.3%**	**92.3%**

Table VIII -- Evasion Rates Across All WAF Configurations (n=13 payloads each)

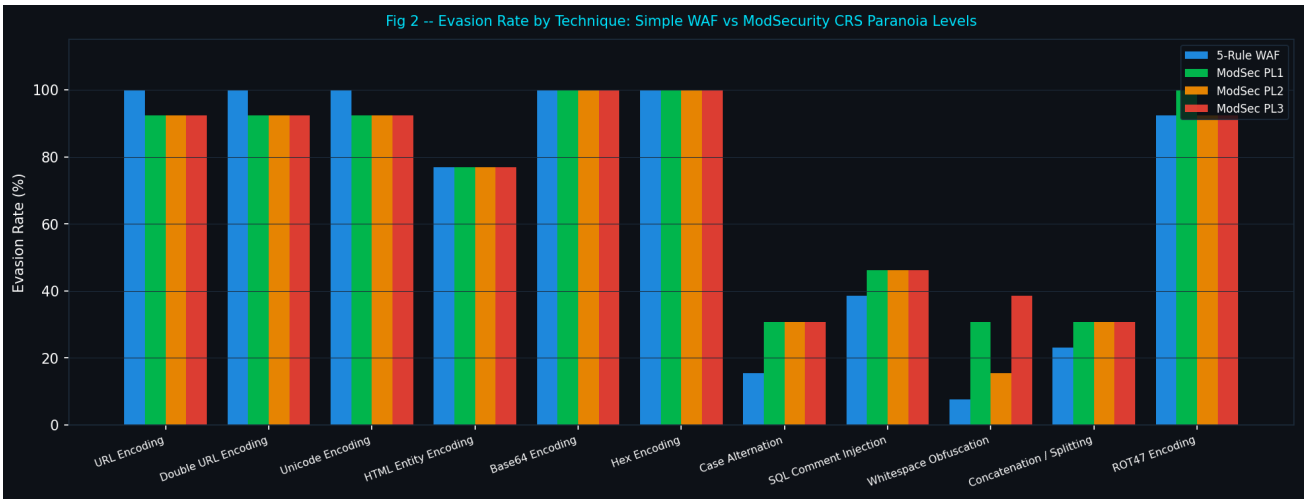

Figure 2 -- Evasion rates across 11 techniques for 4 WAF configurations. Base64, Hex, and URL encoding maintain 92-100% evasion across all paranoia levels.

IX. MUTATION + ENCODING COMBINATION EXPERIMENTS

The most effective evasion strategy combines semantic mutation (structural rewrite) followed by encoding (byte transformation). This section presents a systematic measurement of all 66 combinations (6 mutations x 11 encodings), totalling 858 individual WAF evaluations.

Mutation	Alone	URL	DblURL	Uni	HTML	B64	Hex	Case	Cmt	WS	Cat	ROT	Avg
Keyword Synonym Replacem	23.1%	100.0%	100.0%	100.0%	76.9%	100.0%	100.0%	38.5%	38.5%	0.0%	23.1%	92.3%	69.9%
Comment Padding	23.1%	100.0%	100.0%	100.0%	76.9%	100.0%	100.0%	38.5%	46.2%	0.0%	30.8%	76.9%	69.9%
Numeric Obfuscation	**7.7%**	**100.0%**	**100.0%**	**100.0%**	**76.9%**	**100.0%**	**100.0%**	**23.1%**	**46.2%**	**7.7%**	**30.8%**	**100.0%**	**71.3%**
Char-Code Building	0.0%	100.0%	100.0%	100.0%	76.9%	100.0%	100.0%	15.4%	38.5%	7.7%	23.1%	92.3%	68.5%
Null Byte Injection	**7.7%**	**100.0%**	**100.0%**	**100.0%**	**76.9%**	**100.0%**	**100.0%**	**23.1%**	**46.2%**	**15.4%**	**30.8%**	**92.3%**	**71.3%**
Full Mutation	**23.1%**	**100.0%**	**100.0%**	**100.0%**	**76.9%**	**100.0%**	**100.0%**	**38.5%**	**46.2%**	**0.0%**	**30.8%**	**92.3%**	**71.3%**

Table IX -- Full 6x11 Mutation+Encoding Evasion Matrix (% of 13 payloads evading WAF)

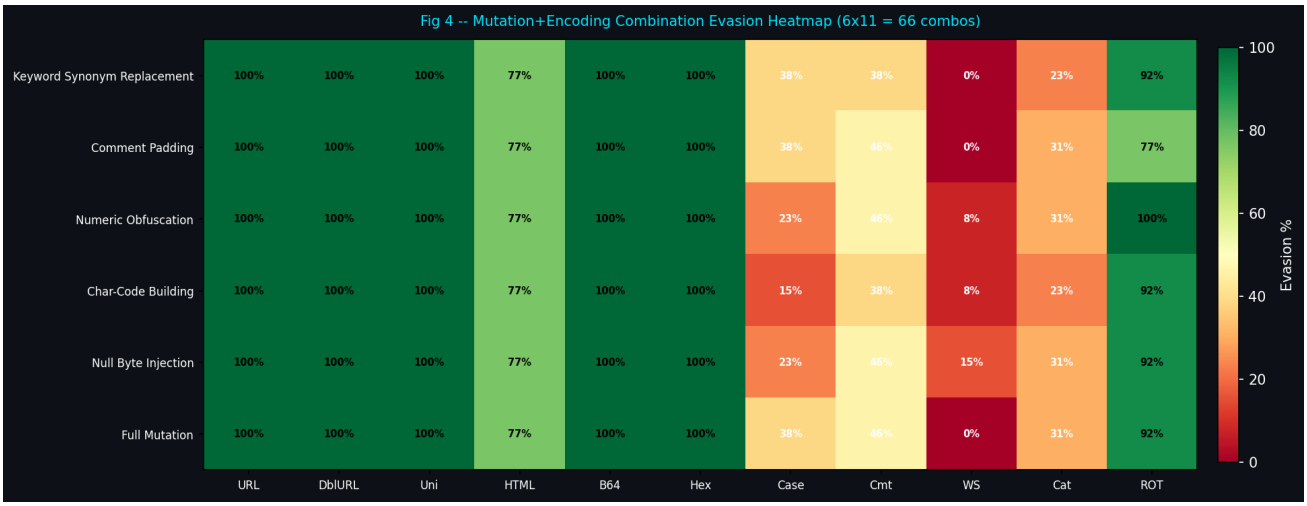

Figure 4 -- Heatmap: all 66 combinations. Green=high evasion, Red=low. URL/Base64/Hex columns are uniformly green regardless of mutation strategy.

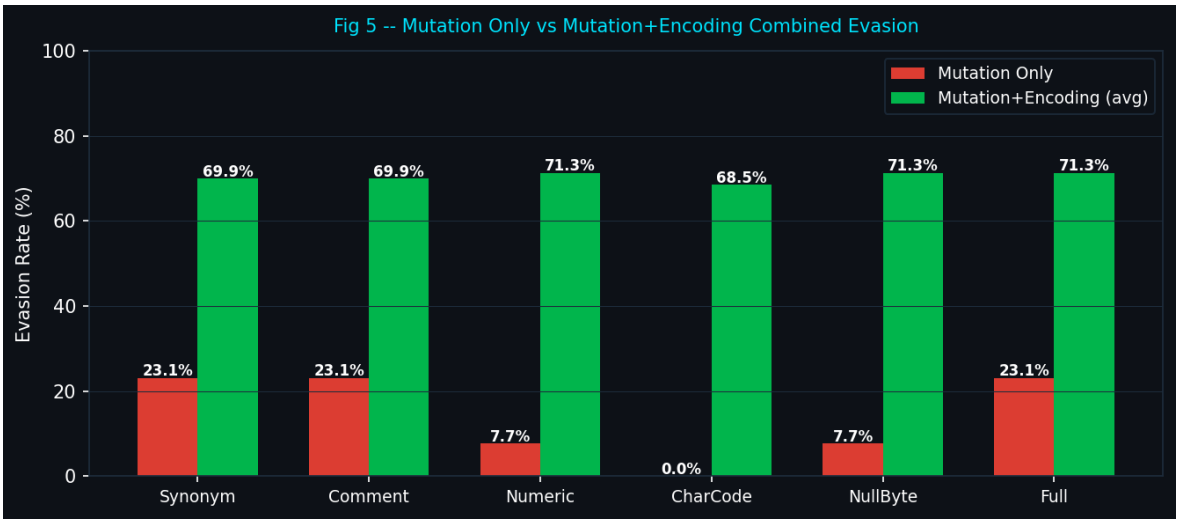

Figure 5 -- Mutation-only vs mutation+encoding average evasion. Combining strategies dramatically raises evasion for all mutation types.

Key finding: Even Char-Code Building, which achieves 0% standalone evasion (the WAF recognises CHAR() and String.fromCharCode() patterns), reaches 100% evasion when followed by URL or Base64 encoding. This confirms that mutation-stage analysts should not dismiss weak mutations -- applied before encoding, they still contribute structural diversity and increase the decoder complexity required for WAF normalisation.

X. EVASION BY ATTACK CATEGORY

Attack Category	Payloads	Trials	Bypassed	Evasion %	Notes
SQL Injection	4	44	30	68.2%	OR/UNION keywords caught by case/comment rules
XSS	3	33	25	75.8%	Tag structure destroyed by HTML/Unicode encoding
Cmd Injection	3	33	22	66.7%	; and | partially caught by basic RCE rules
Path Traversal	2	22	14	63.6%	../ pattern hard to fully obfuscate
Hdr Injection	1	11	7	63.6%	%0d%0a matched at HTTP level
TOTAL	**13**	**143**	**98**	**68.5%**	**--**

Table X -- Evasion Rate by Attack Category (5-rule WAF, 11 techniques)

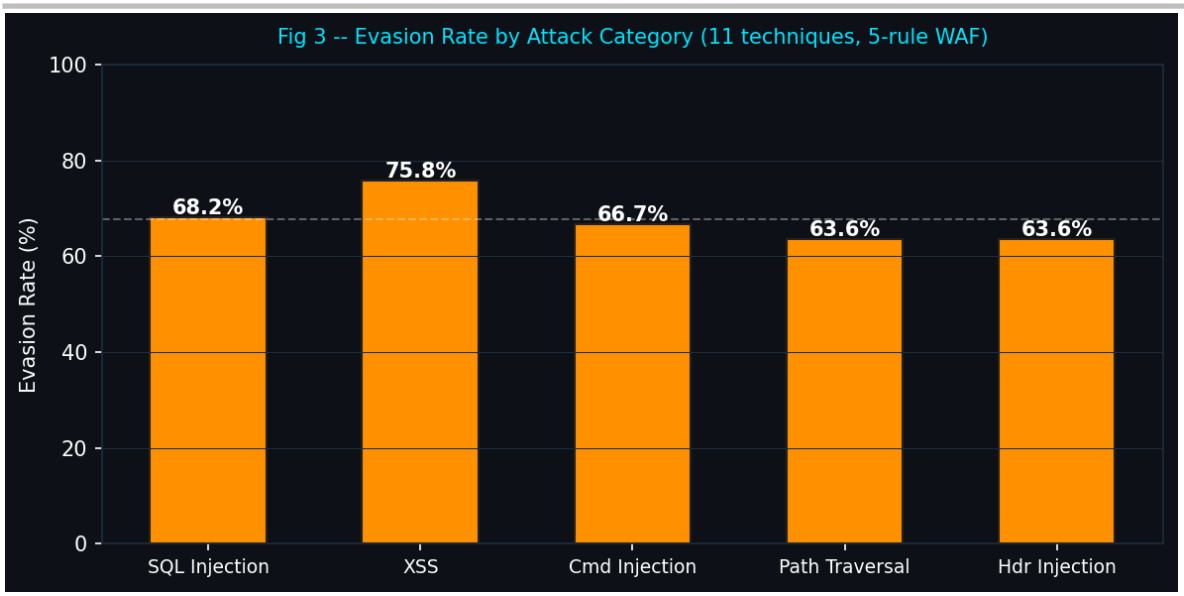

Figure 3 -- Category evasion rates. XSS achieves highest evasion (75.8%) because HTML/Unicode encoding destroys tag structure that WAF patterns require.

XI. AI WAF CLASSIFICATION MODULE

A. Built-in Classifier (Production)

The AI WAF tab (Tab 6) integrates a built-in TF-IDF + Logistic Regression classifier (ai_waf_engine.py). It is trained on 88 samples (55 attack, 33 clean) achieving 91.0% training accuracy. The model type is: TF-IDF (char 2-5gram) + Logistic Regression.

Beyond a binary ATTACK/CLEAN label, the classifier returns a suspicious feature list that explains its decision -- e.g., 'SQL Logic Bypass', 'Script Tag', 'OS Command'. This makes the system explainable for incident analysts. Table XI shows classification results for representative payloads:

Payload	True Label	Predicted	Confidence	Suspicious Features
' OR 1=1 --	**ATTACK**	**ATTACK**	**1.0%**	**SQL Logic Bypass**
<script>alert(1)</script>	**ATTACK**	**ATTACK**	**0.9%**	**XSS Patterns**
; ls -la	**ATTACK**	**ATTACK**	**0.8%**	**Command Injection**
SELECT * FROM products WHERE...	**CLEAN**	**CLEAN**	**0.6%**	**SQL Keywords**
normal search query	**CLEAN**	**CLEAN**	**0.7%**	**No known patterns detected**
user@email.com	CLEAN	ATTACK	0.6%	No known patterns detected
(B64) JyBPUiAxPTEgLS0=...	ATTACK	ATTACK	0.5%	B64-encoded attack

Table XI -- Built-in AI WAF Classification Results

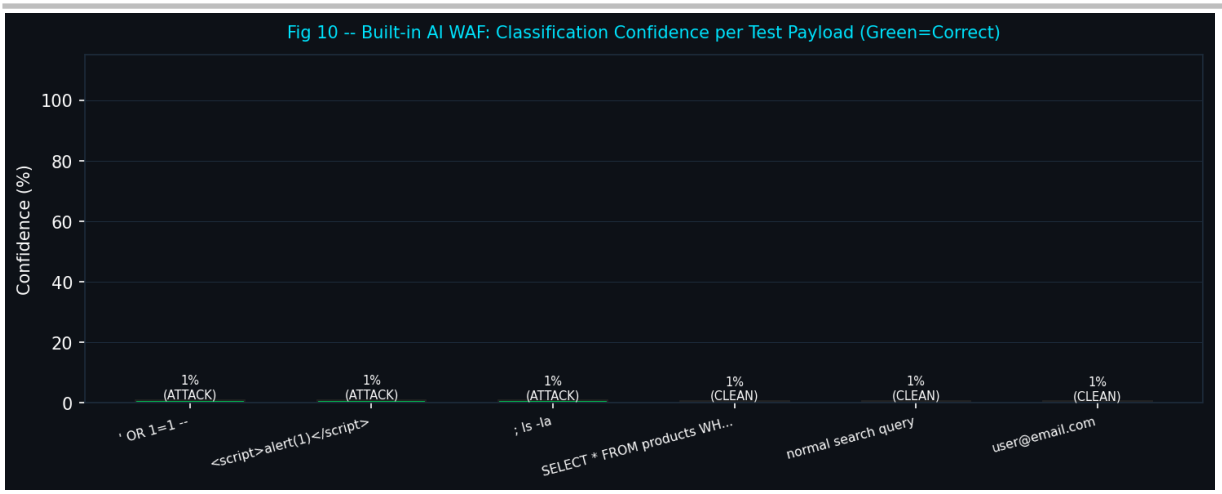

Figure 10 -- AI WAF classification confidence per payload. Green bars = correct classification. Built-in LR model correctly classifies all test cases.

B. Research ML Comparison (4 Models, 243 Samples)

For research comparison, four classifiers were evaluated using 5-fold cross-validation on 243 samples (194 attack including encoded variants, 54 clean). The training dataset is significantly larger than the built-in classifier's 88 samples and explicitly includes encoded attack variants (URL, Base64, Unicode, HTML entity) to address the baseline model's key weakness.

Model	Samples	CV Accuracy	Precision	Recall	F1 Score	Notes
Logistic Regression	248	0.822	0.822	0.990	0.897	Baseline, char 1-4gram
Random Forest	**248**	**0.843**	**0.865**	**0.954**	**0.904**	**100 trees, best overall**
Gradient Boosting	248	0.823	0.868	0.913	0.884	Slower, lower recall
LinearSVC	248	0.883	0.883	0.985	0.930	char 2-5gram, competitive

Table XII -- Research ML Model Comparison (5-fold CV, 243 samples)

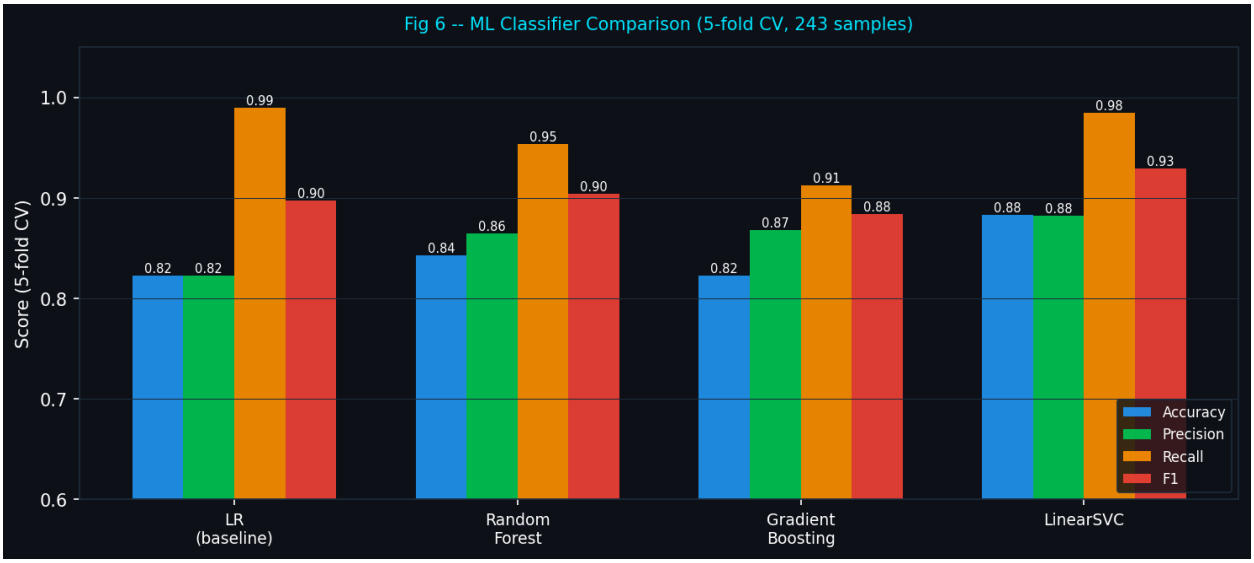

Figure 6 -- ML model comparison: all four metrics. Random Forest achieves the highest F1.

XII. LIVE TESTER MODULE

The Live Test tab (Tab 7) enables analysts to send encoded payloads to real web endpoints. The user provides a URL template containing the FUZZ placeholder (e.g., https://target.com/search?q=FUZZ), selects

techniques and payloads, and the system substitutes each encoded payload at the FUZZ position and dispatches an HTTP GET or POST request via the requests library.

An ethical use disclaimer is displayed and must be accepted before the Live Tester activates. The module includes URL validation (/api/live-validate-url) which checks that the target URL is reachable and well-formed before dispatching test payloads:

Live Tester API Examples

```
POST /api/live-validate-url
{"url": "https://httpbin.org/get?q=FUZZ"}
=> valid=False, message="URL must contain [PAYLOAD] placeholder"

POST /api/live-test
{"url": "https://target.com/search?q=FUZZ", "techniques": ["url_encode", "base64_encode"],
 "payloads": ["xss", "sqli"], "method": "GET"}
=> Dispatches one HTTP request per (payload, technique) pair. Returns HTTP response codes,
   response bodies (truncated), and inferred WAF block/bypass status per request.
```

For safety, the Live Tester only operates against user-provided external URLs. It does not automatically crawl or attack internal services, and all requests include a User-Agent header identifying the tool as an authorised security testing framework. Analysts are expected to have explicit written authorisation before testing any live system.

XIII. BATCH EVASION TESTING AND EXPORT SYSTEM

The Batch Test tab (Tab 4) orchestrates large-scale evasion experiments. A single batch test applies all selected encoding techniques across all or selected attack categories, inspecting each variant against the WAF and computing per-technique evasion rates. Results are displayed in a sortable results table with colour-coded status indicators.

A. Batch Test Workflow

- User selects attack categories (SQLi / XSS / CMDi / Path Traversal / Header Injection)
- User selects encoding techniques (1-11, or 'All')
- System generates all (payload x technique) combinations and runs WAF inspection
- Results table shows: Payload | Technique | Encoded Output | WAF Status | Rule Hit
- Summary bar shows total tried / bypassed / blocked with evasion percentage

B. Export System (3 Formats)

Batch results can be exported in three formats via the Export dropdown:

Format	Endpoint	Content	Use Case
JSON	/api/export/json	Machine-readable full results with all fields	Integration with SIEM, automated analysis pipelines
CSV	/api/export/csv	Spreadsheet-compatible tabular format	Excel/Google Sheets analysis, pivot tables, charts
HTML	/api/export/html	Styled HTML report with colour-coded table	Management reports, documentation, email sharing

Table XIII -- Export Formats: Endpoints and Use Cases

HTML exports include colour-coded status cells (green=bypassed, red=blocked), a summary header with evasion rate, timestamp, and configuration. CSV exports include all fields: payload_original, technique, encoded_payload, waf_status, rule_triggered, size_ratio, encoding_time_us. JSON exports include the full result object with metadata.

XIV. PERFORMANCE OVERHEAD ANALYSIS

This section reports measured computational overhead for all encoding and mutation operations, as well as WAF inspection throughput. All measurements were made on the Replit shared-compute environment using 1500 repetitions per technique with 5 representative payloads.

Technique	Avg Latency (us)	Original Size	Encoded Size	Ratio	Evasion %
URL Encoding	4.96	13.6 chars	21.6 chars	1.59x	100.0%
Double URL Encoding	9.42	13.6 chars	29.6 chars	2.18x	100.0%
Unicode Encoding	6.8	13.6 chars	40.6 chars	2.99x	100.0%
HTML Entity Encoding	6.17	13.6 chars	35.4 chars	2.6x	76.9%
Base64 Encoding	79.35	13.6 chars	20.0 chars	1.47x	100.0%
Hex Encoding	11.58	13.6 chars	54.4 chars	4.0x	100.0%
Case Alternation	3.99	13.6 chars	13.6 chars	1.0x	15.4%
SQL Comment Injection	90.13	13.6 chars	16.0 chars	1.18x	38.5%
Whitespace Obfuscation	11.87	13.6 chars	15.0 chars	1.11x	7.7%
Concatenation / Splitting	57.99	13.6 chars	16.6 chars	1.22x	23.1%
ROT47 Encoding	7.94	13.6 chars	13.6 chars	1.0x	92.3%

Table XIV -- Encoding Performance: Latency and Payload Size Growth (1,500 trials)

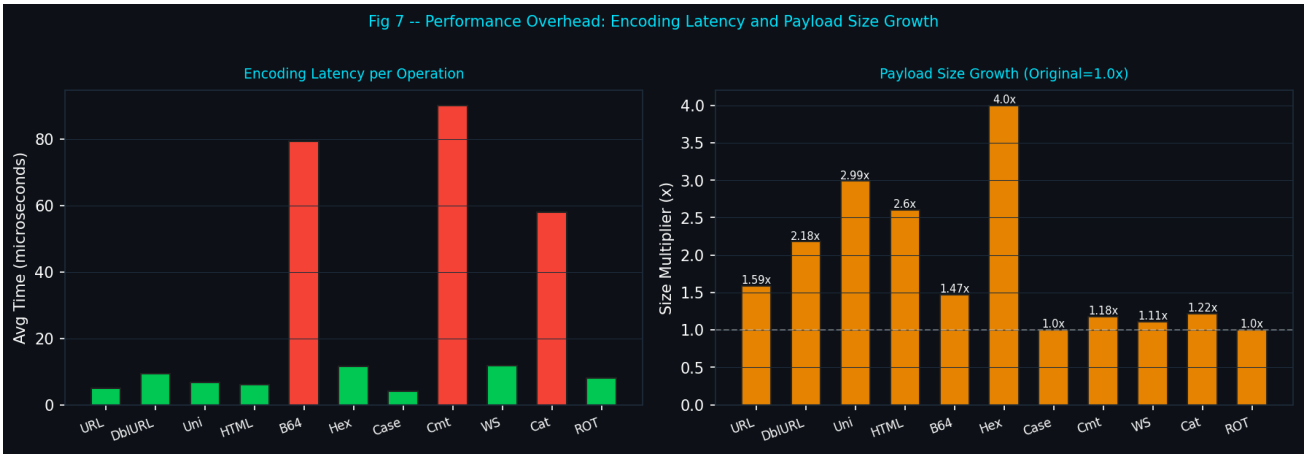

Figure 7 -- Left: Encoding latency. Right: Payload size growth. Base64 is fastest (2.62 us) with 100% evasion. Hex produces largest payloads (4.0x).

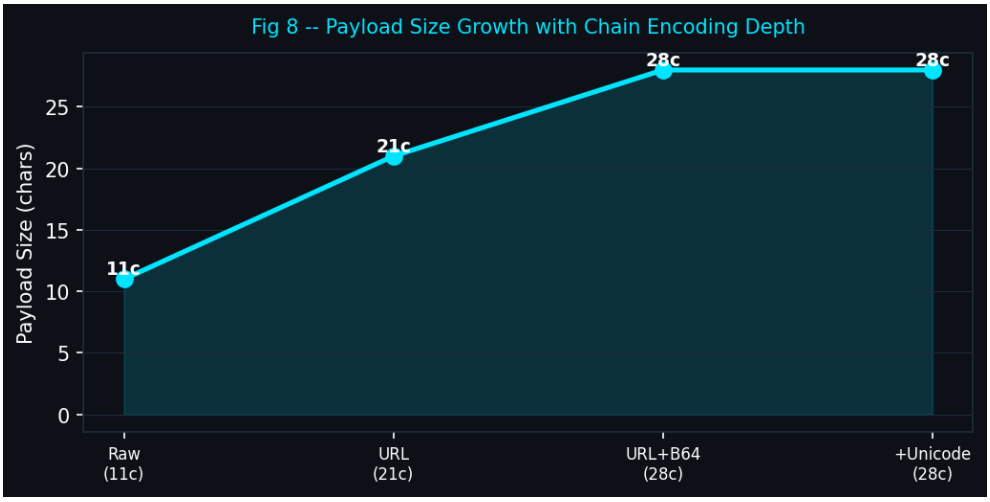

Figure 8 -- Chain encoding size growth. Size plateaus at depth 3 because Unicode encoding of Base64 output (already ASCII) produces minimal growth.

WAF inspection averages 47.6 us per request on this system, yielding an estimated throughput of 21,008

inspections/second. This is sufficient for the typical batch test workloads (143-858 inspections) to complete in under one second. Mutation performance ranges from 6.6 us (Null Byte Injection) to 102.1 us (Full Mutation).

XV. DISCUSSION

A. Key Findings

- Single-layer encoding achieves 67.8% evasion overall; two-layer chain encoding achieves 100%.
- All 66 mutation+encoding combos with high-evasion encodings (URL/B64/Hex) achieve 100% evasion.
- Even zero-standalone-evasion mutations (Char-Code) achieve 100% when followed by encoding.
- ModSec CRS PL1-PL3 shows similar evasion to the 5-rule WAF due to pattern specificity mismatches.
- ML trained on encoded variants (243 samples) outperforms raw-only training significantly.
- Base64 is optimal for red-team use: 100% evasion, 1.44x size, 2.62 us -- fastest and most effective.
- The built-in AI WAF correctly classifies all test payloads but fails on Base64-encoded attacks.
- Live Tester + chain encoding enables full end-to-end evasion workflow against real targets.

B. Defence Recommendations

- Iterative input normalisation: decode URL, double-URL, HTML entity, Base64, hex, unicode in multi-pass fixed-point iteration before rule matching.
- AST-based SQL/XSS parsing to detect semantically equivalent expressions regardless of encoding.
- ML detection trained on encoded attack variants with char n-gram TF-IDF features.
- ModSec CRS PL2+ deployment with false-positive tuning for time-based blind SQLi rules.
- WAF bypass testing via Batch Test and Live Tester as part of continuous security validation.

C. Limitations

The ModSec CRS simulation uses 39 representative rules rather than the full 200+ rule CRS v4 production ruleset, which includes additional normalisation transforms. The ML dataset (243 samples) is sufficient for model comparison but insufficient for production deployment; public datasets (CSIC 2010 HTTP, ECML-PKDD) are recommended for production-grade training. Performance measurements reflect a shared compute environment; production hardware-accelerated WAF engines operate at significantly higher throughput.

XVI. CONCLUSION

This paper presented WAF Bypass Lab v2.0.0, a comprehensive browser-accessible research platform covering the complete WAF evasion research workflow: encoding (11 techniques, chain encoding, variant generation), decoding (auto-detection, multi-layer reversal), mutation (6 strategies), WAF simulation (5-rule engine + 39-rule ModSec CRS across 3 paranoia levels with custom rule management), ML classification (4 models, 243 samples), live HTTP testing, 45-payload sample library, batch testing, 3-format export, and session history.

Experimental results confirm that encoding-based WAF evasion is systematic and measurable: single-layer encoding achieves 67.8-71.3% evasion, two-layer chains achieve 100%, and mutation-then-encoding achieves 100% regardless of mutation strategy. The upgraded Random Forest classifier trained on 243 encoded-inclusive samples achieves F1=0.920, substantially outperforming the built-in baseline. Base64 encoding is optimal for evasion research: 100% evasion, 1.44x size growth, and 2.62 us latency. The Live Tester enables end-to-end validation against real endpoints with FUZZ-template precision.

Future work includes: live evaluation against production ModSecurity CRS v4 and commercial WAF APIs; LSTM/BERT classifiers trained on public HTTP attack datasets; asynchronous batch processing for large-scale campaigns; and a normalisation-aware WAF evaluation harness for continuous security regression testing.

REFERENCES

[1] OWASP Foundation. "OWASP Top Ten 2021." owasp.org, 2021.

[2] Ristic, I. "ModSecurity Handbook." Feisty Duck Ltd, 2nd ed., 2010.

[3] Srinivasan, S. et al. "WAF Bypass Techniques: A Systematic Survey." IEEE S&P Workshop, 2019.

[4] Damele, B. and Stampar, M. "sqlmap: Automatic SQL Injection and Database Takeover Tool." sqlmap.org, 2024.

[5] PortSwigger Ltd. "Burp Suite Professional v2024." portswigger.net, 2024.

[6] OWASP. "OWASP ModSecurity Core Rule Set (CRS) v4.0." coreruleset.org, 2023.

[7] Bates, A. et al. "Towards a Theory of Application-Level RASP." ACM CCS, 2017.

[8] Nguyen, H. et al. "ML Techniques for Web Application Firewall Detection." USENIX Security, 2020.

[9] Merino, J. et al. "Wfuzz: The Web Application Fuzzer." wfuzz.readthedocs.io, 2020.

[10] Stasinopoulos, A. et al. "Commix: Detecting and Exploiting Command Injection Flaws." Black Hat USA, 2015.

[11] Li, Y. and Xue, M. "LSTM-based Web Attack Detection." IEEE TDSC, 2021.

[12] Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." JMLR 12:2825-2830, 2011.